Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления

Кафедра информационных технологий автоматизированных систем

ОТЧЕТ
по лабораторной работе №6
по дисциплине
«Современные системы программирования»

Выполнил: ст. гр. 320601
П. Д. Дудников
Р. В. Земляник

Проверила: Ю. О. Герман

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ 3
2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
3 ВЫПОЛНЕНИЕ РАБОТЫ 5
3.1 Структура программы 5
3.2 Задание 1. Параллельные циклы 5
3.3 Задание 2. *Parallel.ForEach* с коллекциями 6
3.4 Задание 3. *PLINQ* для анализа данных 7
3.5 Задание 4. Разделение данных и агрегация 7
3.6 Задание 5. Обработка изображений 8
3.7 Индивидуальное задание. Вариант 2 9
4 КОНТРОЛЬНЫЕ ВОПРОСЫ 12
ЗАКЛЮЧЕНИЕ 14
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 15
ПРИЛОЖЕНИЕ А 16

1 ЦЕЛЬ РАБОТЫ

Цель лабораторной работы – изучить основы параллельного программирования в *C#*, включая библиотеку *Task Parallel Library* (*TPL*), параллельные циклы *Parallel.For* и *Parallel.ForEach*, *PLINQ* (*Parallel LINQ*), методы разделения данных и агрегации результатов.

Для достижения цели в работе решены следующие задачи:

– сравнить последовательный цикл и параллельные циклы с блокировкой и с локальными переменными;

– обработать коллекцию с помощью *Parallel.For* и *Parallel.ForEach* с отменой и поиском;

– выполнить анализ данных с помощью *PLINQ* и сравнить его с последовательным *LINQ*;

– сравнить стратегии разделения данных и способы агрегации результатов;

– выполнить параллельную обработку изображения, представленного двумерным массивом;

– выполнить индивидуальное задание варианта 2 – параллельную фильтрацию коллекций с пользовательскими условиями.

Все задания выполнены в одном консольном приложении *Lab6* для платформы *.NET* 10 на компьютере с процессором *AMD Ryzen* 7 5800*HS* (8 ядер, 16 логических процессоров). Полный текст программы приведен в приложении А.

2 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

Task Parallel Library (*TPL*) – набор типов и *API* в пространстве имен *System.Threading.Tasks*, который упрощает добавление параллелизма и асинхронности в приложения. *TPL* автоматически масштабирует степень параллелизма для использования всех доступных процессоров [1, 2]. Основные понятия *TPL*:

– *Task* – асинхронная операция;
– *Task<TResult>* – асинхронная операция, возвращающая значение;
– параллелизм задач – разделение работы на независимые задачи, выполняемые одновременно.

Параллельные циклы *Parallel.For* и *Parallel.ForEach* распределяют итерации цикла по потокам пула. Они поддерживают автоматическую балансировку нагрузки, локальные для потока данные, отмену через *CancellationToken* и досрочное завершение через *ParallelLoopState.Break()* и *Stop()* [1]. *Parallel.For* работает с диапазоном индексов, а *Parallel.ForEach* – с любой коллекцией *IEnumerable<T>* или с разделителем *Partitioner*.

PLINQ – параллельная реализация *LINQ to Objects*. Метод *AsParallel()* превращает последовательность в *ParallelQuery*, которая разбивает источник на разделы и обрабатывает их на нескольких ядрах [3]. Метод *WithDegreeOfParallelism(n)* ограничивает число потоков, *AsOrdered()* сохраняет исходный порядок элементов ценой дополнительной буферизации.

Разделение (*partitioning*) – стратегия распределения данных между задачами. Различают разделение на непрерывные диапазоны для массивов и списков, блочное разделение для динамической балансировки и хеш-разделение для операций по ключу [1]. Агрегация объединяет частичные результаты: наиболее эффективна схема с локальной для потока переменной, которая накапливает результат без синхронизации, и одной синхронизированной операцией (*lock* или *Interlocked.Add*) в конце работы потока.

3 ВЫПОЛНЕНИЕ РАБОТЫ

3.1 Структура программы

Программа состоит из одного файла *Program.cs*. Код каждого задания методических указаний помещен в отдельный класс *Task1–Task5*, метод *Main* задания переименован в *Run*, а индивидуальное задание реализовано в классе *Variant2*. Точка входа *LabWork_ParallelProgramming.Program.Main* выводит число логических процессоров и меню выбора задания.

В заданиях 4 и 5 используется класс *Partitioner*, поэтому к директивам *using* добавлено пространство имен *System.Collections.Concurrent*, в котором он объявлен. Все замеры выполнены в конфигурации *Release*. Данные генерируются случайно, поэтому значения сумм и найденных позиций меняются от запуска к запуску.

3.2 Задание 1. Параллельные циклы

Для массива из 10 000 000 случайных чисел вычисляется сумма *Math.Sqrt(Math.Log(x* + 1)) последовательно, параллельным циклом с блокировкой на каждой итерации и параллельным циклом с локальными переменными. Результат приведен на рисунке 1.

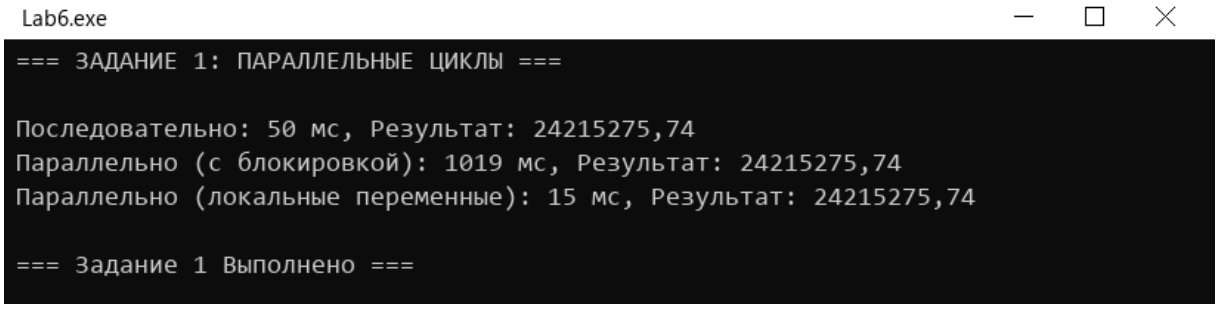


Рисунок 1 – Результат выполнения задания 1

Все три варианта дали одинаковую сумму 24 215 275,74. Последовательный цикл занял 50 мс, вариант с блокировкой – 1019 мс, то есть в 20 раз медленнее, а вариант с локальными переменными – 15 мс. Блокировка выполняется 10 миллионов раз, потоки конкурируют за *lockObj*, часто блокируются и сбрасывают данные синхронизации из кэша, поэтому параллельный код с общей блокировкой оказывается медленнее последовательного. С локальными переменными *lock* вызывается только по одному разу на поток, и параллелизм дает ускорение в 3,3 раза.

3.3 Задание 2. *Parallel.ForEach* с коллекциями

Задание выполняет трансформацию массива из 10 000 000 чисел, фильтрацию через *Parallel.ForEach* с локальными списками, поиск трех значений с отменой, сравнение блокировки и локальных переменных и интерактивный поиск. Результат с вводом чисел 777, 5, 1000 и команды *exit* показан на рисунке 2.

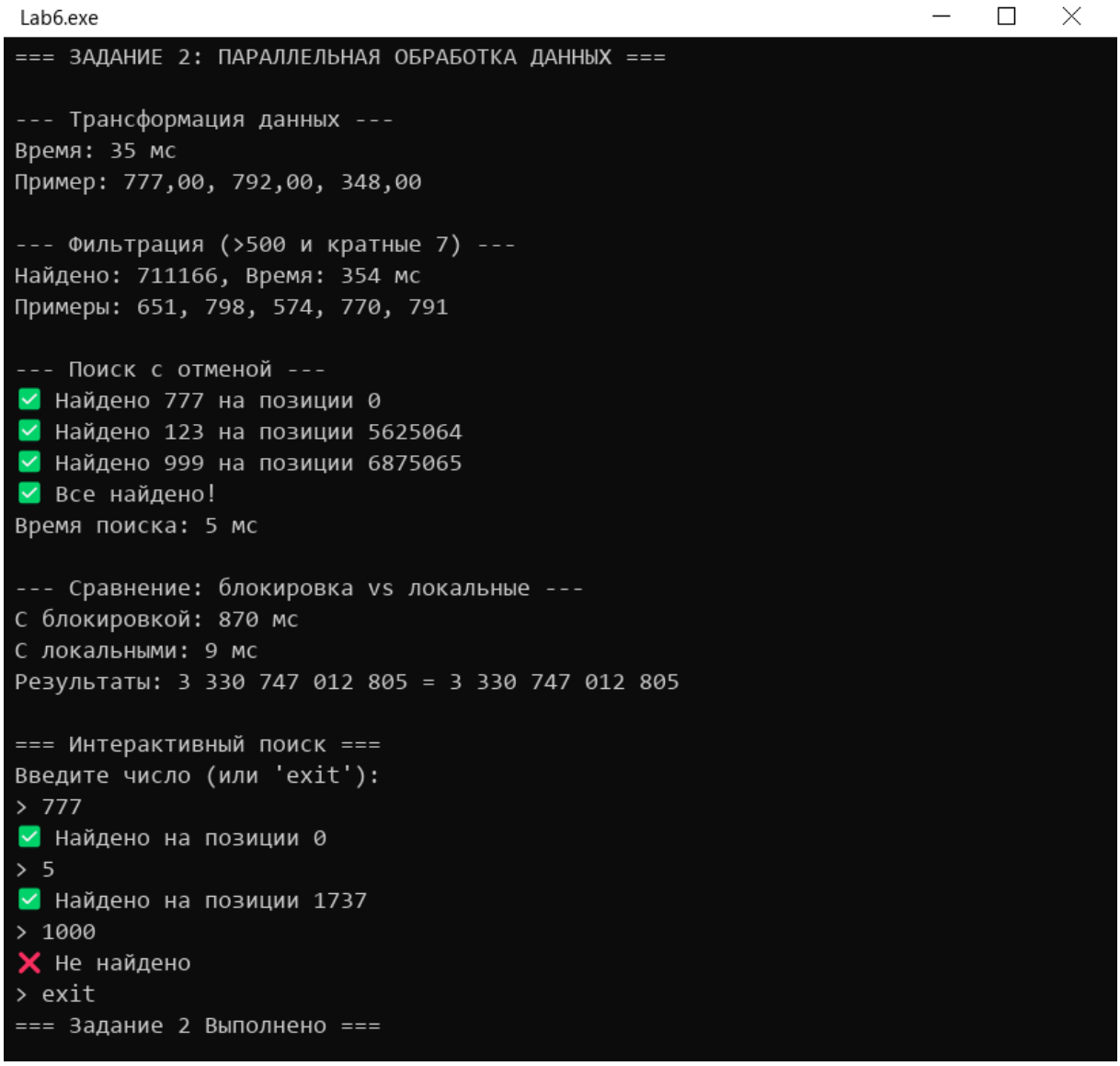


Рисунок 2 – Результат выполнения задания 2

Трансформация заняла 35 мс и не требовала синхронизации, так как каждая итерация пишет в свой элемент массива. Фильтрация нашла 711 166 чисел больше 500 и кратных 7 за 354 мс: основное время уходит на перечисление последовательности *Enumerable.Range(...).Select(...)*, которую *Parallel.ForEach* читает блоками под блокировкой. Поиск с отменой нашел все три значения за 5 мс и завершился исключением *OperationCanceledException*, после которого выведено сообщение «Все найдено!».

Суммирование квадратов с блокировкой заняло 870 мс, с локальными переменными – 9 мс при равных результатах 3 330 747 012 805, то есть локальные переменные дали ускорение почти в 100 раз. Интерактивный поиск нашел числа 777 и 5 и сообщил, что числа 1000 в массиве нет, так как значения генерируются в диапазоне от 1 до 999. Метод *state.Break()* не гарантирует, что найдена первая позиция числа: возвращается позиция, обнаруженная одним из потоков.

3.4 Задание 3. *PLINQ* для анализа данных

Для списка из 10 000 000 чисел вычисляется среднее значение квадратных корней элементов больше 500 с помощью последовательного *LINQ* и *PLINQ* в разных режимах. Результат приведен на рисунке 3.

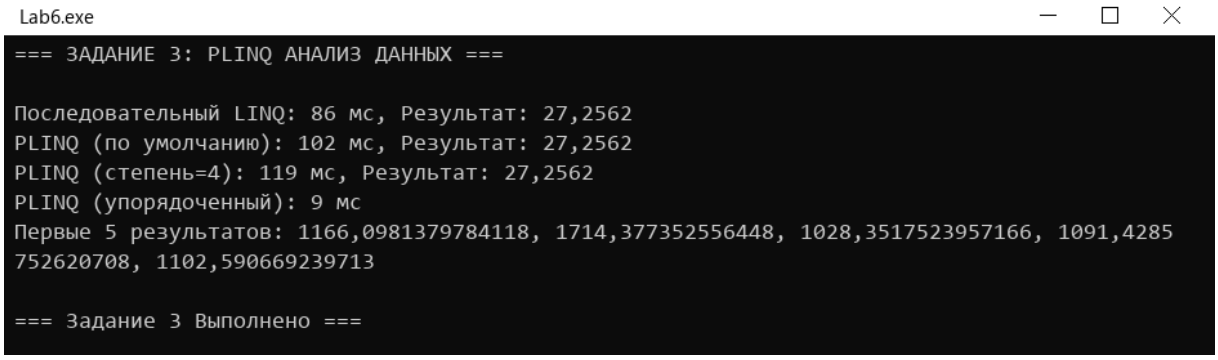


Рисунок 3 – Результат выполнения задания 3

Все варианты вернули одинаковый результат 27,2562. В этом запуске последовательный *LINQ* занял 86 мс, *PLINQ* по умолчанию – 102 мс, *PLINQ* со степенью 4 – 119 мс. Операция *Math.Sqrt* слишком дешевая, поэтому затраты на разбиение источника, запуск потоков и объединение частичных средних сравнимы с самой работой, а первый запуск *PLINQ* дополнительно включает *JIT*-компиляцию его инфраструктуры. Выигрыш от *PLINQ* появляется при затратной обработке элемента, что подтверждает индивидуальное задание. Запрос с *AsOrdered()* и *Take(5)* занял 9 мс, так как остановился после первых пяти результатов и вернул их в исходном порядке.

3.5 Задание 4. Разделение данных и агрегация

Сумма массива из 10 000 000 чисел вычисляется тремя способами: *Parallel.For* с *Interlocked.Add* на каждой итерации, *Parallel.ForEach* по диапазонам *Partitioner.Create(0, N*, 100000) и *Parallel.For* с локальной агрегацией. Результат приведен на рисунке 4 и в таблице 1.

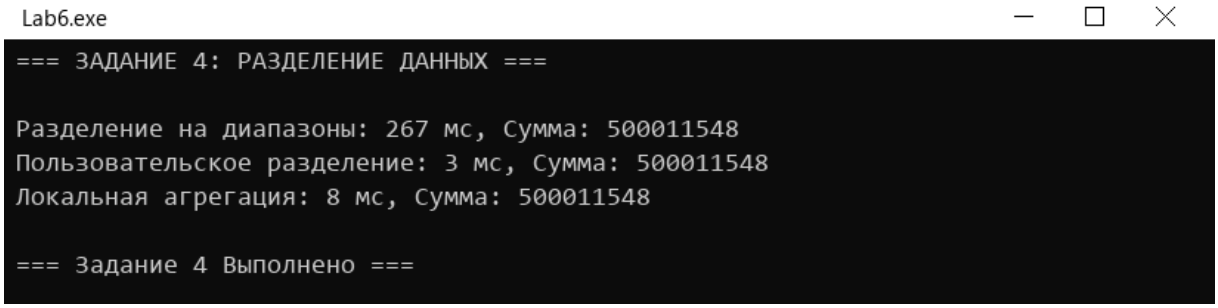


Рисунок 4 – Результат выполнения задания 4

Таблица 1 – Сравнение стратегий разделения и агрегации

Способ	Время, мс	Синхронизаций	Сумма
Разделение на диапазоны, *Interlocked.Add* на каждой итерации	267	10 000 000	500 011 548
Partitioner.Create, блоки по 100 000	3	100	500 011 548
Локальная агрегация в *Parallel.For*	8	по одной на поток	500 011 548

Все способы дали одинаковую сумму. Атомарная операция на каждой итерации заставляет ядра постоянно передавать друг другу строку кэша с переменной *sumRange*, поэтому этот способ самый медленный. Пользовательское разделение быстрее всех: тело делегата вызывается один раз на блок, внутренний цикл по массиву компилируется в обычный быстрый цикл, а *Interlocked.Add* выполняется только 100 раз.

3.6 Задание 5. Обработка изображений

Изображение 2000 на 2000 пикселей представлено двумерным массивом, к каждому пикселю применяется функция *ProcessPixel*. Сравниваются последовательная обработка, *Parallel.For* по строкам, *Parallel.ForEach* с разделением на блоки по 100 строк и *PLINQ*. Результат приведен на рисунке 5.

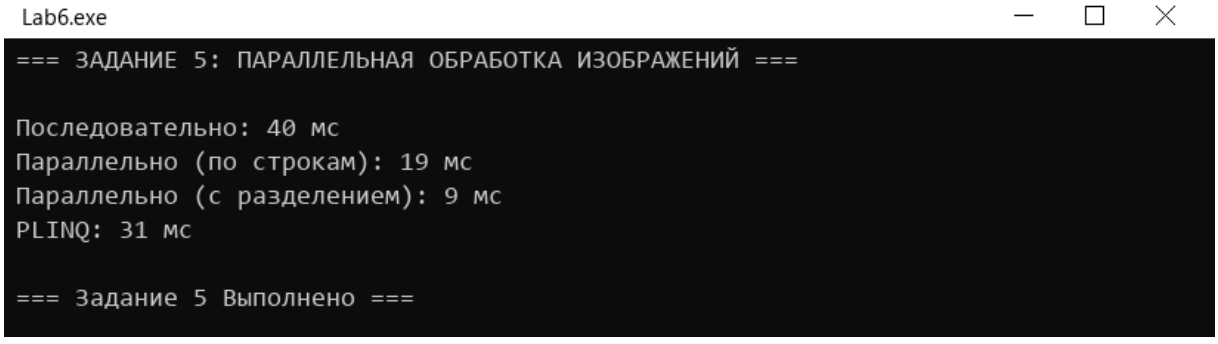


Рисунок 5 – Результат выполнения задания 5

Последовательная обработка заняла 40 мс, *Parallel.For* по строкам – 19 мс, обработка блоками строк – 9 мс, *PLINQ* – 31 мс. Блочное разделение лучше всего, так как поток обрабатывает подряд 100 строк и последовательно проходит память. *PLINQ* медленнее, потому что вычисляет

координаты делением и остатком для каждого из 4 000 000 индексов и собирает результат в новый массив.

3.7 Индивидуальное задание. Вариант 2

Вариант 2 – параллельная фильтрация коллекций с использованием *PLINQ* и пользовательских условий. Коллекция содержит 5 000 000 случайных чисел от 1 до 100 000 000. Условия фильтрации заданы списком делегатов *Func<int, bool>*:

– четные числа – дешевое условие;

– простые числа – затратная проверка делением до квадратного корня;

– простые числа с суммой цифр, кратной 5, – составное условие из двух пользовательских методов.

Для каждого условия сравниваются последовательный *LINQ, PLINQ, PLINQ* с *AsOrdered()* и *Parallel.ForEach* с локальными списками, а результаты проверяются на совпадение. Для самого затратного условия дополнительно измерено влияние степени параллелизма. Результат приведен на рисунке 6 и в таблице 2.

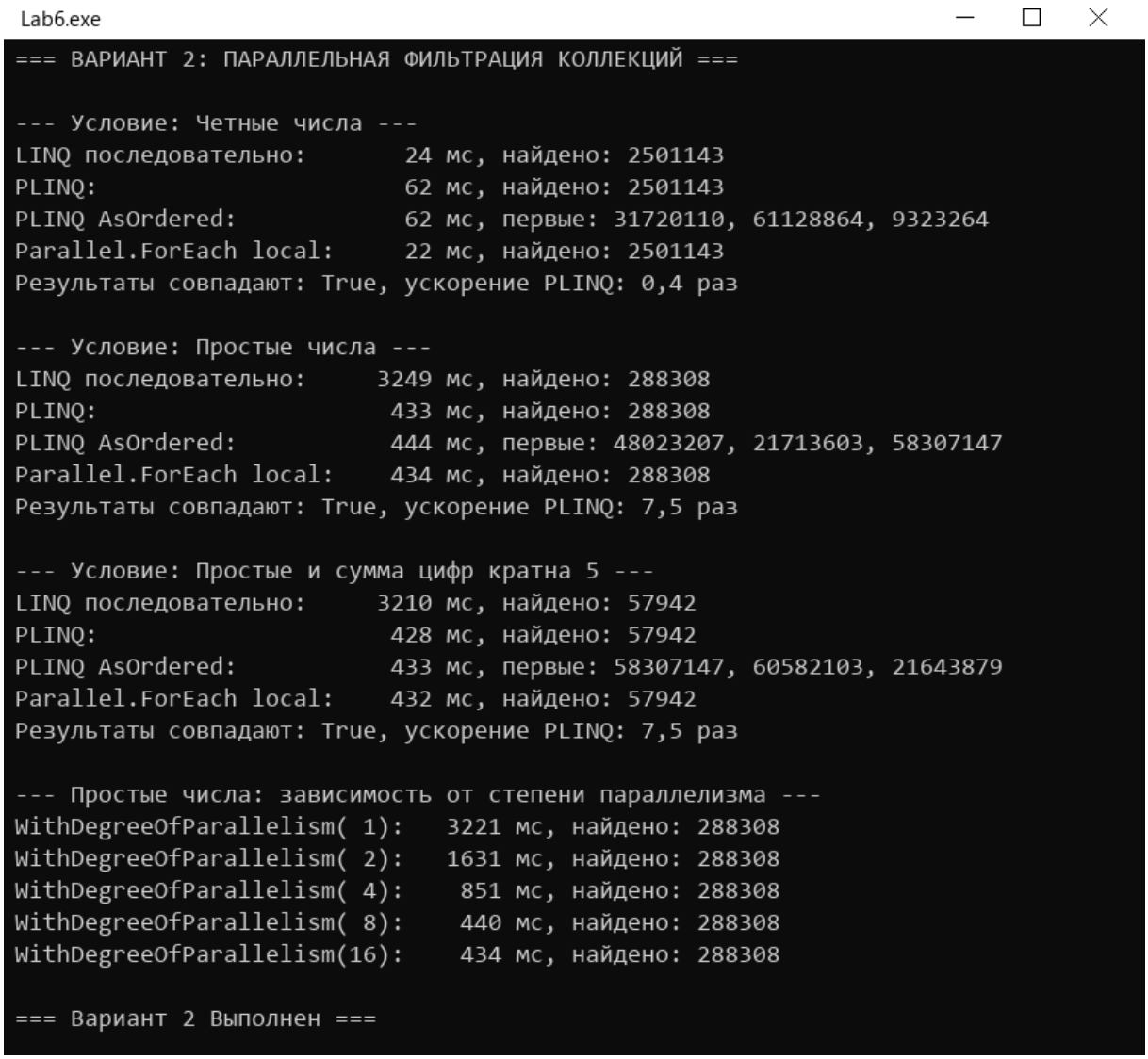


Рисунок 6 – Результат выполнения индивидуального задания

Таблица 2 – Время фильтрации 5 000 000 элементов, мс

Условие	*LINQ*	*PLINQ*	*PLINQ AsOrdered*	*Parallel.ForEach*	Найдено
Четные числа	24	62	62	22	2 501 143
Простые числа	3249	433	444	434	288 308
Простые, сумма цифр кратна 5	3210	428	433	432	57 942

Для дешевого условия *PLINQ* оказался медленнее последовательного *LINQ* (ускорение 0,4), так как проверка четности занимает наносекунды, а основное время уходит на разбиение данных и слияние результатов. Для проверки простоты *PLINQ* ускорил фильтрацию в 7,5 раза при одинаковом количестве найденных чисел, а *AsOrdered()* добавил всего около 10 мс и вернул элементы в исходном порядке.

Зависимость от степени параллелизма для простых чисел приведена в таблице 3.

Таблица 3 – Фильтрация простых чисел при разной степени параллелизма

Степень параллелизма	Время, мс	Ускорение относительно 1 потока
1	3221	1,0
2	1631	2,0
4	851	3,8
8	440	7,3
16	434	7,4

До 8 потоков время уменьшается почти пропорционально их числу, а переход к 16 потокам почти ничего не дает. Процессор *AMD Ryzen* 7 5800*HS* имеет 16 логических процессоров, но они образованы 8 физическими ядрами с технологией одновременной многопоточности *SMT*, и два логических процессора одного ядра делят его исполнительные блоки.

4 КОНТРОЛЬНЫЕ ВОПРОСЫ

Вопрос 1. Что такое *Task Parallel Library* (*TPL*) и каковы ее основные компоненты?

TPL – библиотека пространства имен *System.Threading.Tasks* для параллельного и асинхронного программирования поверх пула потоков. Основные компоненты: классы *Task* и *Task<TResult>* для задач, класс *Parallel* с методами *For, ForEach* и *Invoke*, планировщик *TaskScheduler*, средства отмены *CancellationTokenSource* и *CancellationToken*, а также *PLINQ* и потокобезопасные коллекции *System.Collections.Concurrent*, которые используются вместе с *TPL*.

Вопрос 2. В чем разница между *Parallel.For* и *Parallel.ForEach*?

Parallel.For перебирает целочисленный диапазон индексов и удобен для массивов. *Parallel.ForEach* перебирает элементы любой последовательности *IEnumerable<T>* или разделы, созданные *Partitioner*. Для коллекции без индексов *Parallel.ForEach* вынужден читать элементы перечислителем под синхронизацией, поэтому в задании 2 фильтрация через последовательность заняла больше времени, чем трансформация через *Parallel.For*.

Вопрос 3. Чем *PLINQ* отличается от стандартного *LINQ*?

Стандартный *LINQ* выполняет запрос в одном потоке, *PLINQ* после вызова *AsParallel()* делит источник на разделы и выполняет операторы запроса на нескольких ядрах. По умолчанию *PLINQ* не сохраняет порядок элементов, для этого нужен *AsOrdered()*. *PLINQ* выгоден при затратной обработке элементов, а для дешевых операций может быть медленнее *LINQ*, что видно в задании 3 и в варианте 2.

Вопрос 4. Какие существуют стратегии разделения данных?

Разделение на непрерывные диапазоны подходит для массивов и списков с равномерной работой. Блочное разделение выдает потокам небольшие блоки по мере освобождения и балансирует неравномерную нагрузку. Хеш-разделение распределяет элементы по значению хеш-функции ключа и применяется в операциях группировки и соединения. В *.NET* разделители создаются методом *Partitioner.Create*.

Вопрос 5. Как выполняется агрегация результатов параллельных вычислений?

Каждый поток накапливает частичный результат в локальной переменной, полученной от инициализатора *localInit*, а в делегате *localFinally* объединяет его с общим результатом одной синхронизированной операцией *lock* или *Interlocked.Add*. В *PLINQ* агрегацию выполняют операторы *Sum*,

Average и *Aggregate*. Синхронизация на каждой итерации, как показали задания 1, 2 и 4, замедляет вычисления в десятки раз.

Вопрос 6. Когда следует использовать параллельное программирование вместо последовательного?

Параллелизм оправдан, когда объем данных большой, обработка каждого элемента затратна, итерации независимы и требуют минимальной синхронизации, а компьютер имеет несколько ядер. Для коротких операций, малых коллекций и кода с общей изменяемой переменной накладные расходы превышают выигрыш, поэтому решение принимается по результатам замеров.

ЗАКЛЮЧЕНИЕ

В лабораторной работе изучены параллельные циклы *TPL*, *PLINQ*, стратегии разделения данных и способы агрегации результатов. Код методических указаний выполнен без изменения логики в одном консольном приложении, индивидуальное задание варианта 2 реализовано в том же файле.

Замеры показали, что эффективность параллельного кода определяется синхронизацией и стоимостью работы над элементом. Блокировка на каждой итерации замедлила вычисления в 20 раз по сравнению с последовательным циклом, а локальные переменные и блочное разделение дали наибольшее ускорение. *PLINQ* проигрывает *LINQ* на дешевых операциях, но ускорил фильтрацию простых чисел в 7,5 раза при совпадающих результатах.

Ускорение растет почти линейно до числа физических ядер, после чего прирост прекращается. Абсолютные времена зависят от компьютера и случайных данных, поэтому выводы сделаны по соотношению величин.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Параллельное программирование в *C#* с использованием *TPL, PLINQ* и разделения данных : методические указания к лабораторной работе № 5, часть 2, по дисциплине «Современные системы программирования». – Минск : БГУИР, 2026.

2 Библиотека параллельных задач (*TPL*) [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https://learn.microsoft.com/ru-ru/dotnet/standard/parallel-programming/task-parallel-library-tpl*. – Дата доступа: 22.10.2026.

3 Введение в *PLINQ* [Электронный ресурс] // *Microsoft Learn*. – Режим доступа: *https://learn.microsoft.com/ru-ru/dotnet/standard/parallel-programming/introduction-to-plinq*. – Дата доступа: 22.10.2026.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программы Program.cs

```
// Лабораторная работа 6 (Лабораторная работа 5. Часть 2)
// Параллельное программирование в C# с использованием TPL, PLINQ и
разделения данных
// Все задания собраны в одном файле. Метод Main каждого задания методички
// переименован в Run и вызывается из общего меню.

using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Diagnostics;
using System.Linq;
using System.Text;
using System.Threading;
using System.Threading.Tasks;

namespace LabWork_ParallelProgramming
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;

            while (true)
            {
                Console.WriteLine("\n=== Лабораторная работа 6: Параллельное
программирование (TPL, PLINQ) ===");
                Console.WriteLine($"Логических процессоров:
{Environment.ProcessorCount}");
                Console.WriteLine("1 - Задание 1. Параллельные циклы");
                Console.WriteLine("2 - Задание 2. Parallel.ForEach с
коллекциями");
                Console.WriteLine("3 - Задание 3. PLINQ для анализа данных");
                Console.WriteLine("4 - Задание 4. Разделение данных и
агрегация");
                Console.WriteLine("5 - Задание 5. Комплексная обработка
изображений");
                Console.WriteLine("6 - Вариант 2. Параллельная фильтрация
коллекций");
                Console.WriteLine("0 - Выход");
                Console.Write("> ");

                string choice = Console.ReadLine();
                if (choice == null || choice == "0") break;

                switch (choice)
                {
                    case "1": Task1.Run(); break;
                    case "2": Task2.Run(); break;
                    case "3": Task3.Run(); break;
                    case "4": Task4.Run(); break;
                    case "5": Task5.Run(); break;
                    case "6": Variant2.Run(); break;
                    default: Console.WriteLine("Неизвестный пункт меню");
break;
                }
```

```
            }
        }
    }

    // ==================== Задание 1. Введение в параллельные циклы
====================
    class Task1
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 1: ПАРАЛЛЕЛЬНЫЕ ЦИКЛЫ ===\n");

            // Создание большого массива данных
            const int N = 10000000;
            double[] data = new double[N];
            Random rand = new Random();

            // Инициализация данных
            for (int i = 0; i < N; i++)
            {
                data[i] = rand.NextDouble() * 1000;
            }

            // --- Последовательный цикл ---
            Stopwatch sw = Stopwatch.StartNew();
            double sumSeq = 0;
            for (int i = 0; i < N; i++)
            {
                sumSeq += Math.Sqrt(Math.Log(data[i] + 1));
            }
            sw.Stop();
            Console.WriteLine($"Последовательно: {sw.ElapsedMilliseconds} мс,
Результат: {sumSeq:F2}");

            // --- Параллельный цикл (Parallel.For) ---
            sw.Restart();
            double sumPar = 0;
            object lockObj = new object();
            Parallel.For(0, N, i =>
            {
                double value = Math.Sqrt(Math.Log(data[i] + 1));
                lock (lockObj)
                {
                    sumPar += value;
                }
            });
            sw.Stop();
            Console.WriteLine($"Параллельно (с блокировкой):
{sw.ElapsedMilliseconds} мс, Результат: {sumPar:F2}");

            // --- Параллельный цикл с локальными для потока переменными
(оптимизированный) ---
            sw.Restart();
            double sumParLocal = 0;
            Parallel.For(0, N,
                () => 0.0, // Локальный инициализатор
                (i, state, local) => local + Math.Sqrt(Math.Log(data[i] +
1)), // Тело
                local => { lock (lockObj) { sumParLocal += local; } } //
Агрегатор
            );
            sw.Stop();
```

```
            Console.WriteLine($"Параллельно (локальные переменные):
{sw.ElapsedMilliseconds} мс, Результат: {sumParLocal:F2}");

            Console.WriteLine("\n=== Задание 1 Выполнено ===\n");
        }
    }

    // ====================== Задание 2. Parallel.ForEach с коллекциями
====================
    class Task2
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 2: ПАРАЛЛЕЛЬНАЯ ОБРАБОТКА ДАННЫХ
===\n");

            // Генерация данных
            const int N = 10_000_000;
            int[] numbers = new int[N];
            Random rand = new Random();
            for (int i = 0; i < N; i++) numbers[i] = rand.Next(1, 1000);

            // 1. Трансформация (Parallel.For)
            Console.WriteLine("--- Трансформация данных ---");
            double[] transformed = new double[N];
            Stopwatch sw = Stopwatch.StartNew();
            Parallel.For(0, N, i => transformed[i] =
Math.Sqrt(Math.Pow(numbers[i], 2) + 1));
            sw.Stop();
            Console.WriteLine($"Время: {sw.ElapsedMilliseconds} мс");
            Console.WriteLine($"Пример: {transformed[0]:F2},
{transformed[1000]:F2}, {transformed[5000]:F2}\n");

            // 2. Фильтрация (Parallel.ForEach с локальными переменными)
            Console.WriteLine("--- Фильтрация (>500 и кратные 7) ---");
            List<int> filtered = new List<int>();
            object listLock = new object();
            var ranges = Enumerable.Range(0, N).Select(i => numbers[i]);

            sw.Restart();
            Parallel.ForEach(ranges,
                () => new List<int>(),
                (val, state, local) =>
                {
                    if (val > 500 && val % 7 == 0) local.Add(val);
                    return local;
                },
                local => { lock (listLock) filtered.AddRange(local); }
            );
            sw.Stop();
            Console.WriteLine($"Найдено: {filtered.Count}, Время:
{sw.ElapsedMilliseconds} мс");
            Console.WriteLine($"Примеры: {string.Join(", ",
filtered.Take(5))}\n");

            // 3. Поиск с отменой
            Console.WriteLine("--- Поиск с отменой ---");
            int[] targets = { 777, 999, 123 };
            bool[] found = new bool[targets.Length];
            int[] positions = new int[targets.Length];
            CancellationTokenSource cts = new CancellationTokenSource();
```

```
            sw.Restart();
            try
            {
                Parallel.For(0, N, new ParallelOptions { CancellationToken =
cts.Token }, (i, state) =>
                {
                    for (int j = 0; j < targets.Length; j++)
                    {
                        if (!found[j] && numbers[i] == targets[j])
                        {
                            lock (found)
                            {
                                if (!found[j])
                                {
                                    found[j] = true;
                                    positions[j] = i;
                                    Console.WriteLine($"☑ Найдено
{targets[j]} на позиции {i}");
                                    if (found.All(f => f)) { cts.Cancel();
state.Break(); return; }
                                }
                            }
                        }
                    }
                });
            }
            catch (OperationCanceledException) { Console.WriteLine("☑ Все
найдено!"); }
            sw.Stop();
            Console.WriteLine($"Время поиска: {sw.ElapsedMilliseconds}
мс\n");

            // 4. Сравнение производительности
            Console.WriteLine("--- Сравнение: блокировка vs локальные ---");
            long sumLock = 0, sumLocal = 0;
            object lockObj = new object();

            // С блокировкой
            sw.Restart();
            Parallel.For(0, N, i => { lock (lockObj) sumLock += numbers[i] *
numbers[i]; });
            sw.Stop();
            Console.WriteLine($"С блокировкой: {sw.ElapsedMilliseconds} мс");

            // С локальными переменными
            sw.Restart();
            Parallel.For(0, N,
                () => 0L,
                (i, state, local) => local + numbers[i] * numbers[i],
                local => { lock (lockObj) sumLocal += local; }
            );
            sw.Stop();
            Console.WriteLine($"С локальными: {sw.ElapsedMilliseconds} мс");
            Console.WriteLine($"Результаты: {sumLock:N0} = {sumLocal:N0}\n");

            // 5. Интерактивный поиск
            Console.WriteLine("=== Интерактивный поиск ===");
            Console.WriteLine("Введите число (или 'exit'):");
            while (true)
            {
                Console.Write("> ");
                string input = Console.ReadLine();
                if (input?.ToLower() == "exit") break;
```

```
                if (int.TryParse(input, out int search))
                {
                    bool foundVal = false;
                    int pos = -1;
                    Parallel.For(0, N, (i, state) =>
                    {
                        if (numbers[i] == search) { pos = i; foundVal = true;
state.Break(); }
                    });
                    Console.WriteLine(foundVal ? $"☑ Найдено на позиции
{pos}" : "❌ Не найдено");
                }
            }

            Console.WriteLine("\n=== Задание 2 Выполнено ===");
            Console.ReadKey();
        }
    }

    // ==================== Задание 3. PLINQ для анализа данных
====================
    class Task3
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 3: PLINQ АНАЛИЗ ДАННЫХ ===\n");

            // Генерация большого набора данных
            const int N = 10000000;
            Random rand = new Random();
            var data = Enumerable.Range(0, N)
                .Select(i => rand.NextDouble() * 1000)
                .ToList();

            // --- Последовательный LINQ ---
            Stopwatch sw = Stopwatch.StartNew();
            var seqResult = data
                .Where(x => x > 500)
                .Select(x => Math.Sqrt(x))
                .Average();
            sw.Stop();
            Console.WriteLine($"Последовательный LINQ:
{sw.ElapsedMilliseconds} мс, Результат: {seqResult:F4}");

            // --- PLINQ (базовый) ---
            sw.Restart();
            var plinqResult = data
                .AsParallel()
                .Where(x => x > 500)
                .Select(x => Math.Sqrt(x))
                .Average();
            sw.Stop();
            Console.WriteLine($"PLINQ (по умолчанию):
{sw.ElapsedMilliseconds} мс, Результат: {plinqResult:F4}");

            // --- PLINQ со степенью параллелизма ---
            sw.Restart();
            var plinqResult2 = data
                .AsParallel()
                .WithDegreeOfParallelism(4)
                .Where(x => x > 500)
```

```
                .Select(x => Math.Sqrt(x))
                .Average();
            sw.Stop();
            Console.WriteLine($"PLINQ (степень=4): {sw.ElapsedMilliseconds}
мс, Результат: {plinqResult2:F4}");

            // --- PLINQ с сохранением порядка ---
            sw.Restart();
            var plinqOrdered = data
                .AsParallel()
                .AsOrdered()
                .Where(x => x > 500)
                .Select(x => x * 2)
                .Take(5)
                .ToList();
            sw.Stop();
            Console.WriteLine($"PLINQ (упорядоченный):
{sw.ElapsedMilliseconds} мс");
            Console.WriteLine($"Первые 5 результатов: {string.Join(", ",
plinqOrdered)}");

            Console.WriteLine("\n=== Задание 3 Выполнено ===\n");
        }
    }

    // ==================== Задание 4. Разделение данных и агрегация
====================
    class Task4
    {
        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 4: РАЗДЕЛЕНИЕ ДАННЫХ ===\n");

            const int N = 10000000;
            int[] data = new int[N];
            Random rand = new Random();
            for (int i = 0; i < N; i++)
            {
                data[i] = rand.Next(1, 100);
            }

            // --- Разделение на диапазоны (по умолчанию) ---
            Stopwatch sw = Stopwatch.StartNew();
            long sumRange = 0;
            Parallel.For(0, N, i =>
            {
                Interlocked.Add(ref sumRange, data[i]);
            });
            sw.Stop();
            Console.WriteLine($"Разделение на диапазоны:
{sw.ElapsedMilliseconds} мс, Сумма: {sumRange}");

            // --- Пользовательское разделение с Partitioner ---
            sw.Restart();
            long sumCustom = 0;
            var partitioner = Partitioner.Create(0, N, 100000);
            Parallel.ForEach(partitioner, range =>
            {
                long localSum = 0;
                for (int i = range.Item1; i < range.Item2; i++)
                {
                    localSum += data[i];
                }
```

```
                Interlocked.Add(ref sumCustom, localSum);
            });
            sw.Stop();
            Console.WriteLine($"Пользовательское разделение:
{sw.ElapsedMilliseconds} мс, Сумма: {sumCustom}");

            // --- Локальная для потока агрегация ---
            sw.Restart();
            long sumLocal = 0;
            Parallel.For(0, N,
                () => 0L,
                (i, state, local) => local + data[i],
                local => Interlocked.Add(ref sumLocal, local)
            );
            sw.Stop();
            Console.WriteLine($"Локальная агрегация: {sw.ElapsedMilliseconds}
мс, Сумма: {sumLocal}");

            Console.WriteLine("\n=== Задание 4 Выполнено ===\n");
        }
    }

    // ====================== Задание 5. Комплексная обработка изображений
======================
    class Task5
    {
        // Симуляция функции обработки пикселя
        static double ProcessPixel(double pixel)
        {
            // Сложное вычисление, имитирующее фильтр изображения
            return Math.Sqrt(Math.Abs(Math.Sin(pixel) * Math.Cos(pixel) *
1000));
        }

        public static void Run()
        {
            Console.WriteLine("=== ЗАДАНИЕ 5: ПАРАЛЛЕЛЬНАЯ ОБРАБОТКА
ИЗОБРАЖЕНИЙ ===\n");

            // Симуляция изображения как двумерного массива
            const int width = 2000;
            const int height = 2000;
            double[,] image = new double[height, width];
            double[,] result = new double[height, width];
            Random rand = new Random();

            for (int y = 0; y < height; y++)
            {
                for (int x = 0; x < width; x++)
                {
                    image[y, x] = rand.NextDouble();
                }
            }

            // --- Последовательная обработка ---
            Stopwatch sw = Stopwatch.StartNew();
            for (int y = 0; y < height; y++)
            {
                for (int x = 0; x < width; x++)
                {
                    result[y, x] = ProcessPixel(image[y, x]);
                }
```

```
            }
            sw.Stop();
            Console.WriteLine($"Последовательно: {sw.ElapsedMilliseconds}
мс");

            // --- Параллельная обработка (по строкам) ---
            sw.Restart();
            Parallel.For(0, height, y =>
            {
                for (int x = 0; x < width; x++)
                {
                    result[y, x] = ProcessPixel(image[y, x]);
                }
            });
            sw.Stop();
            Console.WriteLine($"Параллельно (по строкам):
{sw.ElapsedMilliseconds} мс");

            // --- Параллельная обработка с разделением ---
            sw.Restart();
            var partitioner = Partitioner.Create(0, height, 100);
            Parallel.ForEach(partitioner, range =>
            {
                for (int y = range.Item1; y < range.Item2; y++)
                {
                    for (int x = 0; x < width; x++)
                    {
                        result[y, x] = ProcessPixel(image[y, x]);
                    }
                }
            });
            sw.Stop();
            Console.WriteLine($"Параллельно (с разделением):
{sw.ElapsedMilliseconds} мс");

            // --- Обработка с помощью PLINQ ---
            sw.Restart();
            var pixels = Enumerable.Range(0, height * width)
                .AsParallel()
                .Select(i => ProcessPixel(image[i / width, i % width]))
                .ToArray();
            sw.Stop();
            Console.WriteLine($"PLINQ: {sw.ElapsedMilliseconds} мс");

            Console.WriteLine("\n=== Задание 5 Выполнено ===\n");
        }
    }

    // ==================== Вариант 2. Параллельная фильтрация коллекций
====================
    // PLINQ и пользовательские условия фильтрации
    class Variant2
    {
        // Пользовательское условие: число простое (затратная проверка
делением)
        static bool IsPrime(int n)
        {
            if (n < 2) return false;
            for (int d = 2; (long)d * d <= n; d++)
            {
                if (n % d == 0) return false;
            }
            return true;
```

```
        }

        // Пользовательское условие: сумма цифр числа кратна заданному
значению
        static bool DigitSumDivisibleBy(int n, int divisor)
        {
            int sum = 0;
            while (n > 0)
            {
                sum += n % 10;
                n /= 10;
            }
            return sum % divisor == 0;
        }

        public static void Run()
        {
            Console.WriteLine("=== ВАРИАНТ 2: ПАРАЛЛЕЛЬНАЯ ФИЛЬТРАЦИЯ
КОЛЛЕКЦИЙ ===\n");

            // Генерация данных
            const int N = 5_000_000;
            Random rand = new Random();
            List<int> data = Enumerable.Range(0, N)
                .Select(i => rand.Next(1, 100_000_000))
                .ToList();

            // Набор пользовательских условий фильтрации
            var conditions = new List<(string Name, Func<int, bool>
Predicate)>
            {
                ("Четные числа", x => x % 2 == 0),
                ("Простые числа", x => IsPrime(x)),
                ("Простые и сумма цифр кратна 5", x => IsPrime(x) &&
DigitSumDivisibleBy(x, 5))
            };

            foreach (var condition in conditions)
            {
                Console.WriteLine($"--- Условие: {condition.Name} ---");

                // --- Последовательный LINQ ---
                Stopwatch sw = Stopwatch.StartNew();
                List<int> seqResult =
data.Where(condition.Predicate).ToList();
                sw.Stop();
                long seqTime = sw.ElapsedMilliseconds;
                Console.WriteLine($"LINQ последовательно:   {seqTime,6} мс,
найдено: {seqResult.Count}");

                // --- PLINQ (по умолчанию) ---
                sw.Restart();
                List<int> plinqResult =
data.AsParallel().Where(condition.Predicate).ToList();
                sw.Stop();
                long plinqTime = sw.ElapsedMilliseconds;
                Console.WriteLine($"PLINQ:                  {plinqTime,6} мс,
найдено: {plinqResult.Count}");

                // --- PLINQ с сохранением порядка ---
                sw.Restart();
```

```
                List<int> orderedResult =
data.AsParallel().AsOrdered().Where(condition.Predicate).ToList();
                sw.Stop();
                Console.WriteLine($"PLINQ AsOrdered:
{sw.ElapsedMilliseconds,6} мс, первые: {string.Join(", ",
orderedResult.Take(3))}");

                // --- Parallel.ForEach с локальными списками ---
                sw.Restart();
                List<int> forEachResult = new List<int>();
                object listLock = new object();
                Parallel.ForEach(data,
                    () => new List<int>(),
                    (val, state, local) =>
                    {
                        if (condition.Predicate(val)) local.Add(val);
                        return local;
                    },
                    local => { lock (listLock) forEachResult.AddRange(local);
}
                );
                sw.Stop();
                Console.WriteLine($"Parallel.ForEach local:
{sw.ElapsedMilliseconds,6} мс, найдено: {forEachResult.Count}");

                bool same = seqResult.SequenceEqual(orderedResult) &&
plinqResult.Count == seqResult.Count
                            && forEachResult.Count == seqResult.Count;
                Console.WriteLine($"Результаты совпадают: {same}, ускорение
PLINQ: {(double)seqTime / Math.Max(plinqTime, 1):F1} раз\n");
            }

            // --- Степень параллелизма для самого затратного условия ---
            Console.WriteLine("--- Простые числа: зависимость от степени
параллелизма ---");
            for (int degree = 1; degree <= Environment.ProcessorCount; degree
*= 2)
            {
                Stopwatch sw = Stopwatch.StartNew();
                int count =
data.AsParallel().WithDegreeOfParallelism(degree).Count(x => IsPrime(x));
                sw.Stop();
                Console.WriteLine($"WithDegreeOfParallelism({degree,2}):
{sw.ElapsedMilliseconds,6} мс, найдено: {count}");
            }

            Console.WriteLine("\n=== Вариант 2 Выполнен ===\n");
        }
    }
}
```